JSPM's Bhivarabai Sawant Institute of Technology and Research

Wagholi, Pune – 412207

Department of Computer Engineering

MINI PROJECT REPORT

Design and Develop a Tool for Digital Forensic of Images

Submitted in partial fulfillment of the requirements for the mini project in

Cyber Security and Digital Forensics
Laboratory Practice-IV

Submitted To:
Dr. Phadtare Tushar

Submitted By:
Shivshankar Mali
Division: C
Roll No.: 49

Academic Year: 2026–27
Semester: VII

Abstract

Digital images are frequently used in communication, documentation, and investigations. Image files can be edited, recompressed, or stripped of metadata, so a systematic first-level examination can be useful.

This mini project presents an Image Forensics Tool developed in Python. The application displays image properties, extracts available EXIF metadata, calculates a SHA-256 hash, and generates an Error Level Analysis (ELA) preview. A graphical user interface makes the analysis easy to perform.

ELA is used only as a preliminary screening aid; it does not independently prove that an image has been manipulated.

Contents

1 Introduction **1**

1.1 Motivation 1

1.2 Objectives 1

2 Technology Stack **2**

3 System Architecture **3**

3.1 Architecture Overview 3

3.2 Architecture Diagram 3

3.3 Main Modules 4

4 Implementation **5**

4.1 Project Structure 5

4.2 SHA-256 Hash Generation 5

4.3 Error Level Analysis 6

4.4 User Interface 6

5 Output and Screenshots **7**

5.1 Main Application Output 7

5.2 ELA Analysis Output 8

5.3 Observed Results 8

5.4 Testing 9

6 Limitations and Forensic Considerations **10**

7 Project Setup and Execution **11**

7.1 Prerequisites 11

7.2 Installation 11

7.3 Execution Steps 11

8 Conclusion **12**

8.1 Future Scope 12

References **13**

List of Figures

3.1 System architecture of the Image Forensics Tool 3

5.1 Image Forensics Tool showing the selected image and forensic report . . . 7
5.2 Error Level Analysis output for the sample image 8

Chapter 1

Introduction

Digital image forensics is the process of examining image files to understand their properties, integrity, metadata, and possible signs of editing. Simple visual inspection may not reveal all changes made to an image.

This project combines basic image information, file hashing, EXIF metadata inspection, and Error Level Analysis in one desktop application.

1.1 Motivation

- **Image information:** Displays format, dimensions, colour mode, and file size.
- **Integrity checking:** SHA-256 can be used to compare the exact contents of files.
- **Metadata inspection:** Shows EXIF information when it is available in the image.
- **Preliminary analysis:** ELA highlights differences between an image and a recompressed copy.
- **Simple interface:** A graphical application makes the tool easier to use.

1.2 Objectives

1. Develop a graphical tool to select and examine images.
2. Display basic image properties and available metadata.
3. Calculate a SHA-256 hash of the selected file.
4. Generate an ELA preview for preliminary inspection.
5. Allow the ELA result to be saved as an image.
6. Document the implementation and actual application output.

Chapter 2

Technology Stack

Component	Technology	Purpose
Programming language	Python	Application logic
Graphical interface	Tkinter	Buttons and report panel
Image processing	Pillow	Read images and generate ELA
File integrity	hashlib / SHA-256	Generate file hash
Metadata	Pillow EXIF support	Read available EXIF tags
Report preparation	LaTeX / Overleaf	Technical documentation

Table 2.1: Technology stack used in the project

Chapter 3

System Architecture

3.1 Architecture Overview

The user selects an image through the Tkinter interface. The application reads its properties and available EXIF metadata, computes the SHA-256 hash, and creates an ELA preview.

The information is displayed in the forensic report panel. The ELA result can also be saved as a PNG file.

3.2 Architecture Diagram

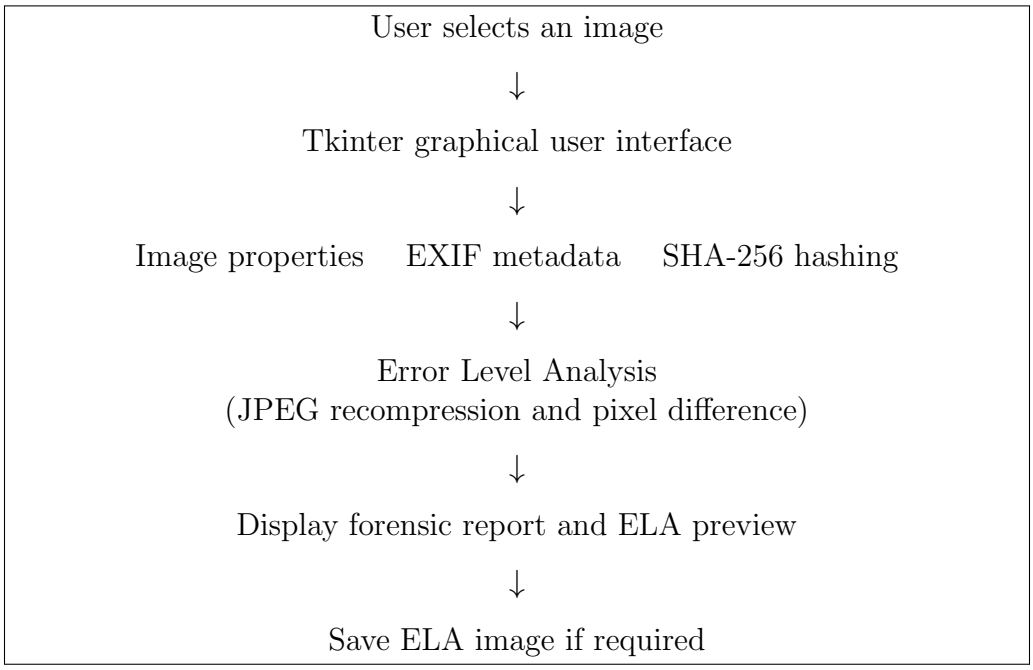


Figure 3.1: System architecture of the Image Forensics Tool

3.3 Main Modules

- **Image Loader:** Opens a selected image file.
- **Image Property Analyzer:** Displays format, dimensions, colour mode, and size.
- **Metadata Analyzer:** Reads available EXIF fields.
- **Hash Generator:** Calculates the SHA-256 digest of the original file.
- **ELA Processor:** Compares the image with a recompressed JPEG copy.
- **Report Viewer:** Shows analysis results and provides the save option.

Chapter 4

Implementation

4.1 Project Structure

The project folder contains the following main files:

```
Image_Forensics_Mini_Project/
|-- app.py
|-- requirements.txt
|-- README.md
|-- main.tex
|-- logo.png
|-- 1.png
|-- 2.png
```

4.2 SHA-256 Hash Generation

The program reads the image file in binary chunks and calculates its SHA-256 digest. This digest can be compared with a later digest to check whether the file bytes have changed.

Listing 4.1: SHA-256 hash generation

```
@staticmethod
def sha256_file(path):
    hasher = hashlib.sha256()

    with open(path, "rb") as file:
        for chunk in iter(
            lambda: file.read(1024 * 1024), b""
        ):
            hasher.update(chunk)

    return hasher.hexdigest()
```

4.3 Error Level Analysis

The application converts the image to RGB, creates a JPEG-recompressed copy, calculates the pixel difference, and enhances the difference for display.

Uneven areas may deserve further examination, but ELA results can also be affected by normal compression, resizing, and editing.

Listing 4.2: ELA processing outline

```
image = source.convert("RGB")
image.save(buffer, format="JPEG", quality=90)

recompressed = Image.open(buffer).convert("RGB")
difference = ImageChops.difference(image, recompressed)

ela_result = ImageEnhance.Brightness(
    difference
).enhance(scale)
```

4.4 User Interface

The interface contains three main areas: the selected image preview, the ELA analysis preview, and the forensic report.

The buttons allow the user to choose an image, save the ELA result, or clear the current analysis.

Chapter 5

Output and Screenshots

5.1 Main Application Output

Figure 5.1 shows the running application after the sample image was selected. The original image, ELA preview, and forensic report are visible in the same window.

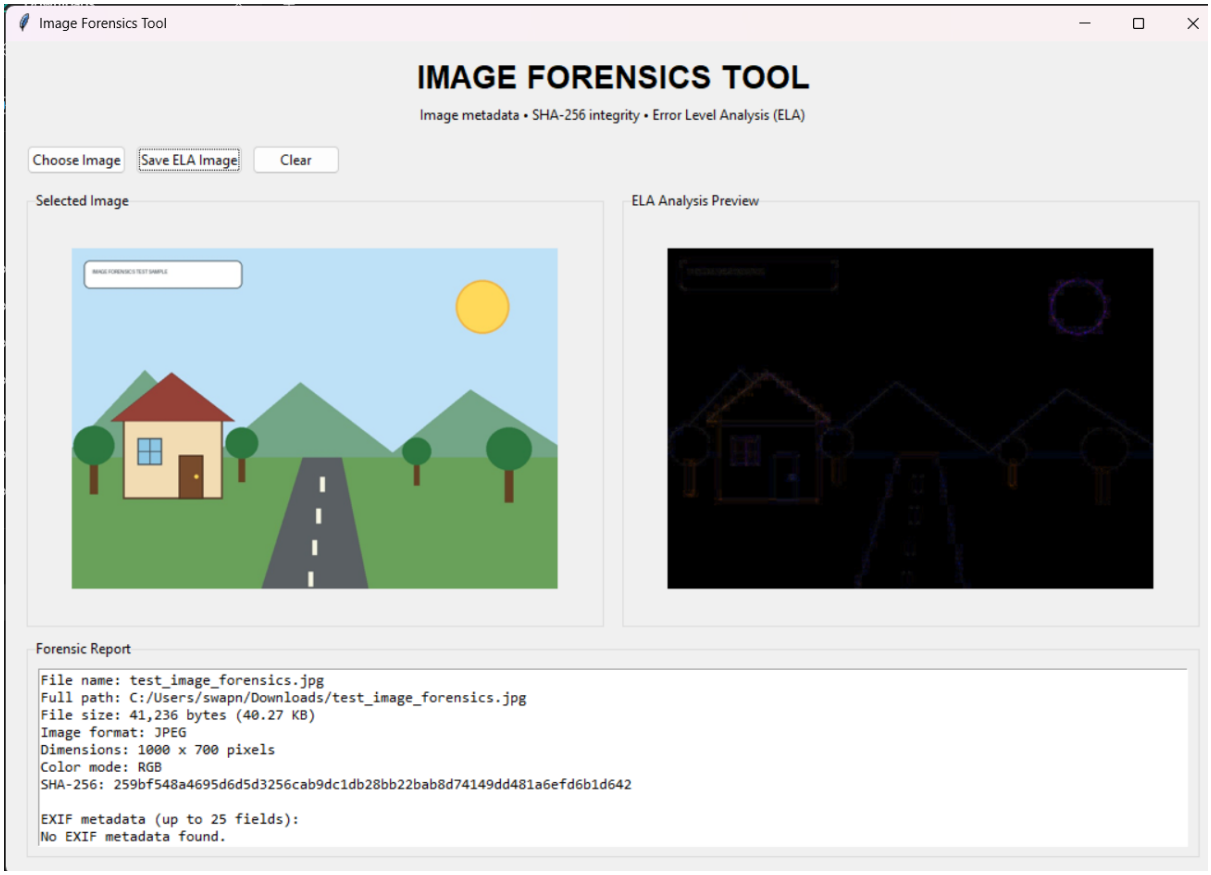


Figure 5.1: Image Forensics Tool showing the selected image and forensic report

5.2 ELA Analysis Output

Figure 5.2 shows the ELA result saved or displayed by the application. The enhanced difference image makes some edges and regions more visible for closer inspection.

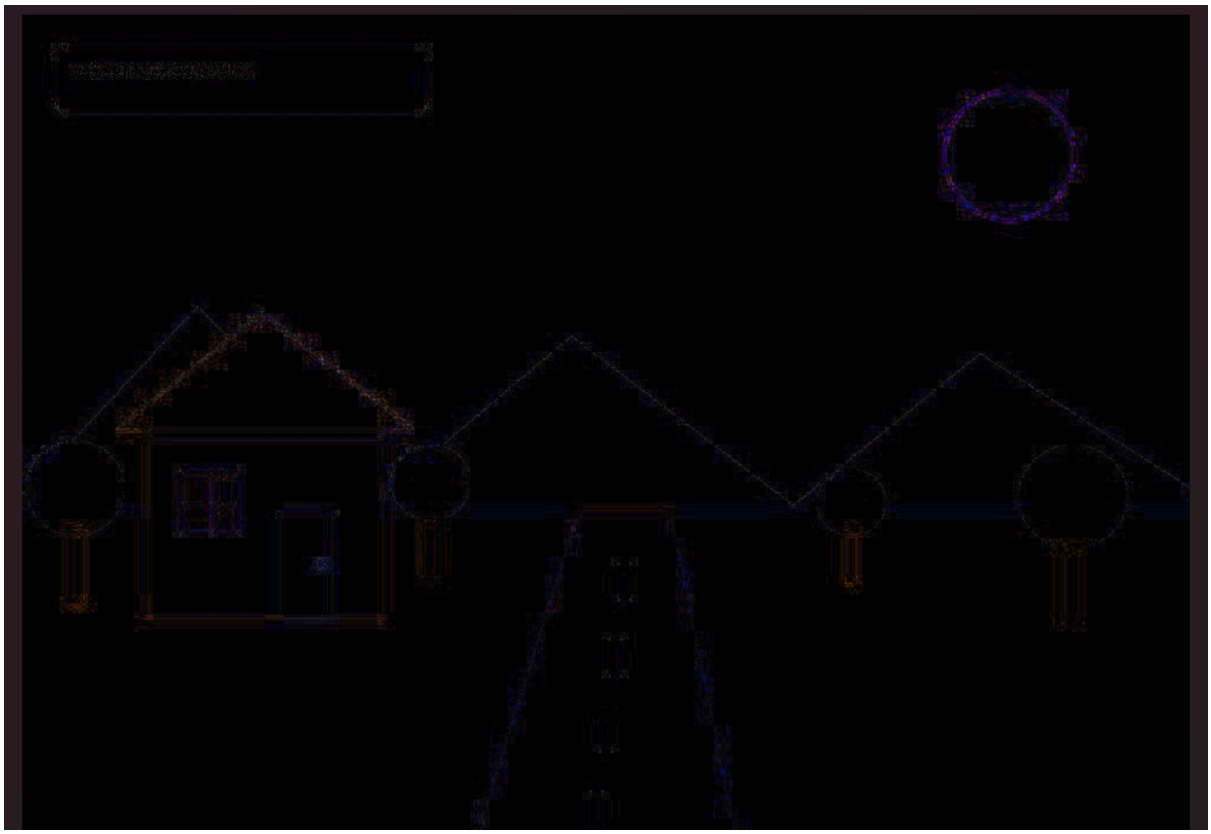


Figure 5.2: Error Level Analysis output for the sample image

5.3 Observed Results

For the sample image used during testing, the application displayed the file name, path, file size, image format, dimensions, colour mode, and SHA-256 digest.

The report indicated that no EXIF metadata was found in the sample file. The ELA preview was generated successfully.

Property	Observed output
File name	`test_image_forensics.jpg`
File size	41,236 bytes (40.27 KB)
Image format	JPEG
Dimensions	1000 × 700 pixels
Colour mode	RGB
EXIF metadata	No EXIF metadata found
SHA-256	Generated and displayed by the application
ELA preview	Generated successfully

Table 5.1: Observed output for the sample image

5.4 Testing

Test case	Expected result	Status
Select a valid image	Image preview is displayed	Pass
Read image properties	Format, dimensions, and size are displayed	Pass
Generate SHA-256	Hash is displayed	Pass
Read EXIF metadata	Metadata or a no-metadata message is displayed	Pass
Generate ELA	ELA preview is displayed	Pass
Save ELA image	PNG file is saved to the selected location	Verify

Table 5.2: Testing summary; verify the save test after performing it

Chapter 6

Limitations and Forensic Considerations

- ELA is a preliminary screening method, not a definitive image forgery detector.
- Compression, resizing, and ordinary editing can create uneven ELA patterns.
- EXIF metadata may be missing, removed, or modified.
- SHA-256 verifies whether file bytes match; it does not prove that the scene shown is genuine.
- Findings should be supported by additional evidence before drawing forensic conclusions.

Chapter 7

Project Setup and Execution

7.1 Prerequisites

- Python 3.9 or later
- Pillow Python library
- Tkinter, usually included with the standard Windows Python installation

7.2 Installation

Open a terminal in the project folder and run:

```
python -m pip install -r requirements.txt
```

7.3 Execution Steps

1. Open the project folder in VS Code.
2. Install the required Pillow library.
3. Run the program using `python app.py`.
4. Click *Choose Image* and select an image.
5. Review the image properties, SHA-256 hash, EXIF metadata, and ELA preview.
6. Click *Save ELA Image* to save the analysis result when required.

Chapter 8

Conclusion

The Image Forensics Tool provides a simple desktop interface for preliminary image examination. It displays image properties, available EXIF metadata, a SHA-256 hash, and an ELA preview.

The project demonstrates how Python image-processing libraries and hashing functions can be combined in a useful forensic utility. ELA results should be interpreted carefully and must not be treated as conclusive proof of image manipulation.

8.1 Future Scope

- Add export of the forensic report to PDF.
- Add perceptual hashing to identify visually similar images.
- Add side-by-side comparison of two images.
- Add a timestamped evidence-handling log.
- Evaluate more forensic techniques on a labeled dataset.

References

1. Python Documentation. https://docs.python.org/3/
2. Pillow Documentation. https://pillow.readthedocs.io/
3. Python hashlib Documentation. https://docs.python.org/3/library/hashlib.html
4. Python Tkinter Documentation. https://docs.python.org/3/library/tkinter.html
5. Krawetz, N. (2007). A Picture's Worth: Digital Image Analysis and Forensics.